

ELTE IK - Computer Science BSc

Final Exam Topics

17. Operating systems

Note: This document was translated from Hungarian to English by AI. Please verify the information against the original Hungarian source before relying on it.

Operating systems

Implementation of processes, their scheduling algorithms. Concurrency, critical sections, implementation of mutual exclusion. Peterson's algorithm. Semaphores, shared memory, message passing. Scheduling options of input and output devices, deadlocks. Memory management, the concept of virtual memory management. Paging and segmentation. Page replacement algorithms (e.g.: LRU). Storage devices, redundancy, file systems, their basic types and services, characteristics.

1 Operating system

A program system that controls the execution of programs in the computer system: for example it schedules the execution of programs, distributes the resources, ensures the communication between the user and the computer system. A program that provides a simple user interface, hiding the devices of the computer (system).

2 Implementation of processes, their scheduling algorithms

2.1 Implementation of processes

Program: a set of bytes in the file system.

Process: a running program in memory (code + I/O data + state).

How many processes operate at once?

In the case of true multitasking the processes go completely independently of each other. Operating systems follow a sequential model, which is pseudo-multitasking. Here at a given moment only one process accesses the memory, and it switches quickly between things (multiprogramming).

For the operating system to be able to handle these processes correctly, it has to supervise the processes. Our operating system thus keeps track of every single process, and with the help of the scheduler, also called the soul of the operating system, it gives, nicely in turn, every single process a small processor (CPU) time slice, during which the given process works, that is, can get onto the processor. *System model:* 1 processor + 1 system memory + 1 I/O device = 1 task execution

In interactive (windowed) systems several programs, processes run.

- Context-switching system: only the foreground application runs
- Cooperative system: the current process, at certain intervals, or at a time-critical operation, voluntarily gives up the CPU (Win 3.1)

- Preemptive system: the kernel takes the control from the current process after a certain time, and gives it to the next waiting process.
- Real-time system: an operating system should provide the possibility of taking the time factor into account. Today's OSes are starting to be supplemented with such properties too.

Creation of processes: today we typically use preemptive systems. The reason for creation can be: system initialization, a system call resulting in a process (a copy of the original [fork], replacement of the original [execve]), user request (command&), batch tasks of large systems.

Processes can run in the foreground, or in the background. The latter we call daemons.

Relationship of processes: Parent-child relationship, process tree: a process has one parent, but a process can have several children; there are related process groups.

Reincarnation server: starter of driver programs, servers; if one of them dies, then it re-creates it, reincarnates it.

Termination of processes: A process, after its start, (carries out) its task within the given time frame. The termination can be voluntary or involuntary.

- Voluntary terminations: Regular exit (exit, return, etc.), Exit due to some error that the program discovers (likewise, e.g., with the return statement).
- Involuntary terminations: Illegal instruction, fatal error (division by 0, use of non-existent memory, etc.), With external help: Another process, or we "shoot" down the given process.

State of processes: A process is an independent program unit, with its own instruction counter, stack, etc. Generally the processes are not independent; the activity of one or another depends on the result of another. A process can be in three states:

- Running
- Ready to run, temporarily stopped, waiting for the scheduler to give CPU time to the process
- Blocked, if the activity cannot logically be continued, because e.g. it waits for the result of another. (cat-Fradi.txt|grepFradi|sort, grep and sort blocked at the beginning)



Figure 1: State transitions

Further states: sleeping, stopped, zombie (if a child process finishes, but its parent does not call wait(&st), then the child stays in the process table; the init process deletes these).

Implementation of processes: The processor only executes the current instructions. At a time one process is active. The processor knows nothing about processes. If we replace the active process with the next one, then

everything has to be preserved so that we can return to the continuation. The “everything”: instruction counter, registers, allocated memory state, open file infos, etc. We store this data in the so-called process descriptor table (process table, process control block). There is also an I/O interrupt vector.

We can also classify processes by whether it is a computation-intensive or input/output-intensive (i.e. I/O) process. It is easy to see that for computation-intensive processes it is best if the scheduler generally gives them the processor for a long time period, while for input/output-intensive processes the shorter period time is more appropriate.

2.2 Scheduling algorithms

Process switching: Can be initiated by a timer, interrupt, event, system call initiation. The scheduler saves the current process characteristics into the process descriptor table. It loads the state of the next process, and the processor continues the work. The caches cannot be saved. Frequent switching requires extra resources. The “good” specification of the process switching time is not unambiguous.

Process control block (PCB): created at system initialization; 1 element, the system bootstrap is already in it when the system starts. An array-like structure (based on PID), but each element is a structure containing complex process data. The more important data of a process: Its identifier (ID), name (program name); Owner, group identifier; Memory, register data, etc.

Threads: Independently operating program units (thread), a separate instruction sequence within a process. Generally one process = one instruction sequence = one thread. Sometimes it can be necessary for there to be “several instruction sequences” within a process. The multi-instruction-sequence process is often called a “lightweight process”. Within a process there can be several mutually “independent” execution sequences. There can be one thread within a process or several threads within a process. In the latter case, if a thread is blocked, the process is also blocked. There is a separate thread table.

A process has an independent address space, global variables, opened file descriptors, child processes, signal handlers, alarms, etc.; a thread does not have these; but both have instruction counters, registers, and a stack.

Only 1 process can run at a time. The scheduler makes the decision. Based on a scheduling algorithm it decides which one should run.

There is certainly a process switch: if a process finishes, or if a process gets into a blocked state (due to I/O or a semaphore). Generally there is a switch: if a new process is created, an I/O interrupt occurs (after an I/O interrupt typically a blocked process that waited for this can continue its run), a timer interrupt (non-preemptive scheduling, preemptive scheduling).

Classification of scheduling:

- Characteristic of every system is fairness: that everyone can access the CPU, the same principles apply to everyone, and they get the “same” load.
- Batch systems: Throughput, turnaround time, CPU utilization
- Interactive systems: Response time, conformance to user needs
- Real-time systems: Meeting deadlines, avoiding data loss, quality degradation

Scheduling in batch systems:

- Sequential scheduling, non-preemptive
 - First Come First Served - (FCFS)
 - A process runs until it finishes, or it blocks.
 - If it blocks, it gets to the end of the queue.
 - Fair, simple, we keep the processes in a linked list.
 - Its disadvantage: I/O-intensive processes finish very slowly.

- Shortest job first, also non-preemptive (shortest job first – SJF)
 - The run times have to be known in advance
 - It is optimal if initially everyone is available
- Shortest remaining run time next
 - Preemptive, examination at every new entry.
- Three-level scheduling
 - Admission scheduler: It selectively admits tasks into memory.
 - Disk scheduler: If the admission scheduler admits many processes and the memory runs out, then some have to be written to disk, and back. This runs rarely.
 - CPU scheduler: We can choose from among the previously mentioned algorithms.

Scheduling in interactive systems:

- Round Robin scheduling
 - Everyone gets a time slice, at the end of which, or in case of blocking, the next process comes
 - At the end of the time slice the next one in the circular list becomes the current process
 - Fair, simple
 - We can store the processes (their characteristics) in a list, and go around and around on this.
 - The size of the time slice can be a problem, because process switching is time-consuming. If the time is small, then a lot of CPU goes to switching, and if it is too large, then to interactive users e.g. keyboard handling can seem slow.
- Priority scheduling
 - Introduction of importance, priority. Unix: 0-49 non-preemptible (kernel) priority, 50-127 user priority
 - The highest priority can run. Dynamic priority modification, otherwise starvation
 - Use of priority classes. Within a class, Round Robin. The priority of the processes has to be adjusted, otherwise the low ones get to the CPU very rarely. Typically at every 100 time slices the priorities are re-evaluated, and then typically the high priorities get lower, then RR goes on this row. At the end the original classes are set up again.
- Multiple queues
 - Likewise priority-based and RR
 - At the highest level every process gets 1 time slice
 - The next 2, then 4, 8, 16, 32, 64.
 - If the highest-level process used up its time, it gets one level lower.
- Shortest process first
 - Although we do not know the remaining time, we estimate it from the previous ones.
 - Aging, weighted average for the time slice: T_0 , $T_0/2+T_1/2$, $T_0/4+T_1/4+T_2/2$, $T_0/8+T_1/8+T_2/4+T_3/2$
- Guaranteed scheduling
 - Every active process gets a proportional CPU time.
 - It has to be tracked how much time a process has already gotten; if someone got proportionally less time, that gets ahead.

- Lottery scheduling
 - Like the previous, only we distribute “lottery tickets” among the processes, and the one who holds the drawn ticket gets the control
 - Proportional CPU time is easy to ensure, useful e.g. for video servers
- Fair-share scheduling
 - Let us also take the users into account. Like the guaranteed one, only related to the users.

Scheduling in real-time systems

In a real-time system time is a key player. It has to be guaranteed that the activity, response is given by a given deadline.

- Hard Real Time (strict), absolute, non-modifiable deadlines.
- Soft Real Time (tolerant), the deadlines exist, but small-degree missing of them is tolerable.

The programs are broken into several smaller processes. Upon detecting an external event, a response is needed by a given deadline. Schedulable: if the sum of the CPU response times of n events per unit time ≤ 1 .

The presence of child processes in the system is common. The parent does not necessarily need the same priority for every one of its children. Typically the kernel uses *priority scheduling* (+RR):

- It provides a system call with which the parent can give the priority of the child
- The kernel schedules – the user process specifies the principle, priority.

Thread scheduling:

- User-level threads
 - The kernel does not know about them; the process gets a time slice, and within this the thread scheduler decides which one should run
 - Fast switching between threads
 - Application-dependent thread scheduling is possible
- Kernel-level threads
 - The kernel knows the threads, the kernel decides which thread of which process comes next
 - Slow switching, between the switch of two threads a full context switch is needed
 - This is also taken into account.

3 Concurrency, critical sections, implementation of mutual exclusion

3.1 Concurrency and its implementation

The scheduler “creates” the feeling of parallel execution by quickly switching the processes.

In multiprocessor systems there are several processors in one machine, the performance is greater, but it generally does not increase reliability.

Clusters: the main goal is primarily increasing reliability.

3.2 Critical sections

Those instructions, that part of the program in which our program uses the common data, are called the critical area, critical section, or critical block.

Key question: the use of common resources, when two processes use the same memory. The critical program area, section, is the part when we use the common resource (memory).

Race condition: two or more processes write or read common memory, the end result depends on the run-time moment. It causes hard-to-discover errors.

Solution: A method that ensures that the common data can be used by only one process at a time.

3.3 Mutual exclusion and its implementation

By mutual exclusion we mean the method that ensures that if a process uses some shared, common data, then other processes cannot access it during this time.

A good mutual exclusion conforms to the following conditions:

- No two processes are in their critical section at the same time.
- There is no speed, CPU parameter dependency.
- No process outside the critical section may block another process.
- No process waits forever to be able to enter the critical section.

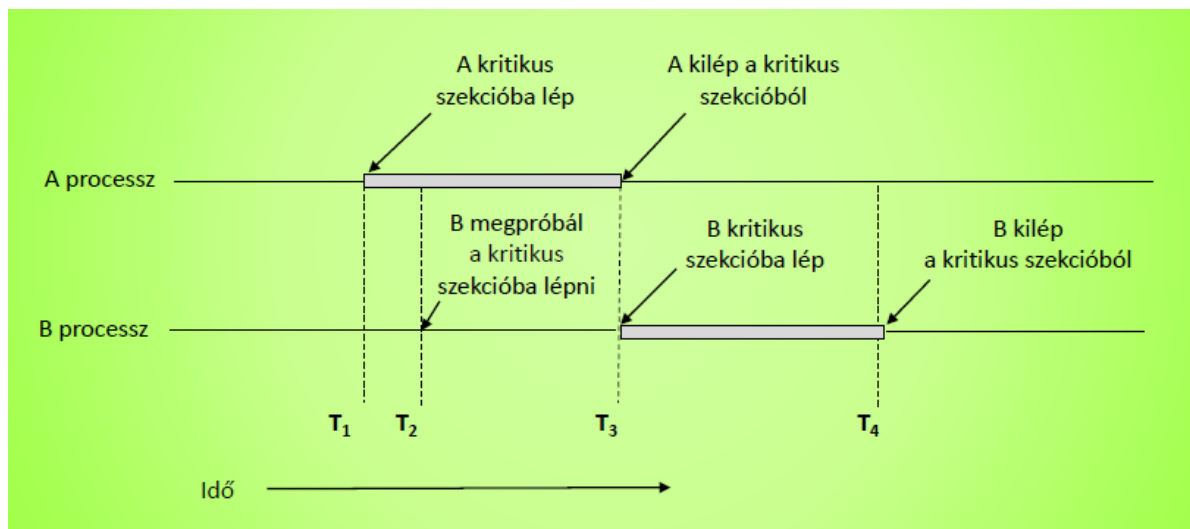


Figure 2: The required behavior of mutual exclusion

3.3.1 Implementations

- Disabling interrupts (all): At entry, disabling all interrupts; at exit, enabling them. This is not really good, since the disabling of interrupts would be in the hands of the user processes; of course the kernel uses it.
- Use of a shared, so-called lock variable: 0 (nobody) and 1 (somebody) is in the critical section. Two processes can also get into the critical section! One process enters the critical section, but just before setting it to 1 the other process gets scheduled.
- Strict alternation: Can also be generalized to several processes. It satisfies the conditions of mutual exclusion except for the 3rd, since if, e.g., 1 process is in the slow, non-critical section, and the 0 process quickly enters the critical section, then also finishes the non-critical section, then this process blocks because the next=1! (It blocks itself!)

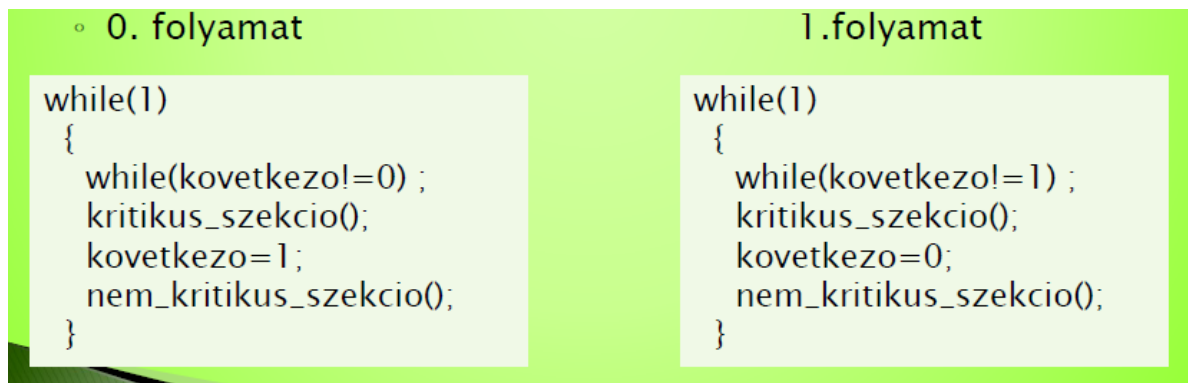


Figure 3: Strict alternation

- G. L. Peterson improved strict alternation. Before the critical section every process calls the enter, then after it the leave function.

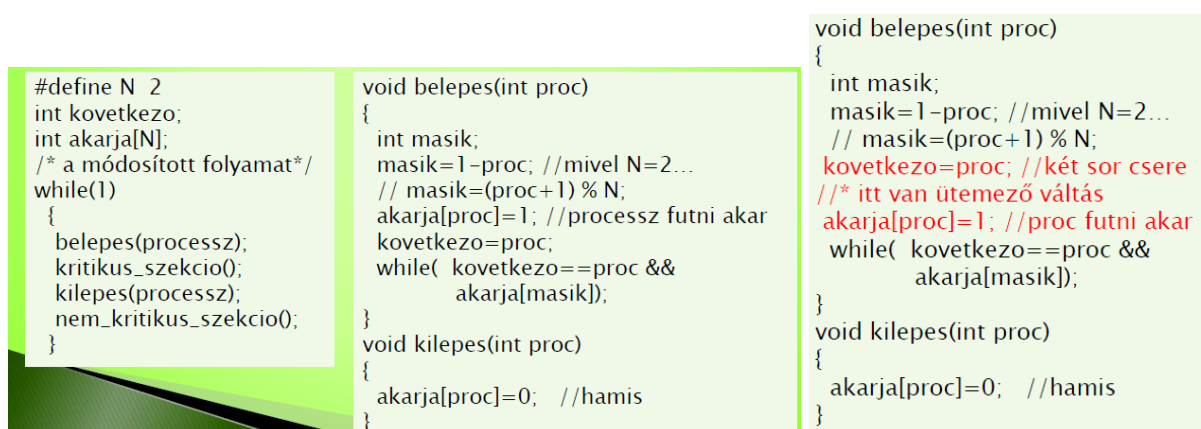


Figure 4: Peterson's improvement and the error hidden in it

This improvement, however, can cause a big error. Suppose $\text{proc}=0$. At the marked scheduling switch the entry of $\text{proc}=1$ comes. Since the value of $\text{wants}[0]$ is 0, therefore process 1 enters the critical section! Then the scheduler switches again, $\text{wants}[1]=1$, the value of next is likewise 1, so $\text{next}==\text{proc}$ is false, that is, process 0 also enters the critical section!

- Busy waiting in machine code: TSL instruction, Test and Set Lock. This is an atomic operation, that is, non-interruptible.
- Busy waiting: The earlier Peterson solution, and the use of TSL, are also good, only we wait in a loop. The earlier solutions we call solved with busy waiting (active waiting), because we run the CPU in an “empty” loop during the waiting. But this wastes CPU time. Instead it would be better if, upon entering the critical section, the process would block if it is not allowed to enter. Active waiting is not really efficient. Solution: instead of waiting, block (sleep) the process, then when permitted, wake it up. They can also be implemented with different parameter specifications. A typical problem: the Producer-Consumer problem or, otherwise, the bounded-buffer problem. E.g.: the Baker-bakery-Customer triangle:
 - The baker bakes the bread as long as there is space on the shelves of the bakery.
 - The customer can buy if there is bread on the shelves of the bakery.
 - If the bakery is full of bread, then “the baker goes to rest”.
 - If the bakery is empty, then the customer waits for bread.

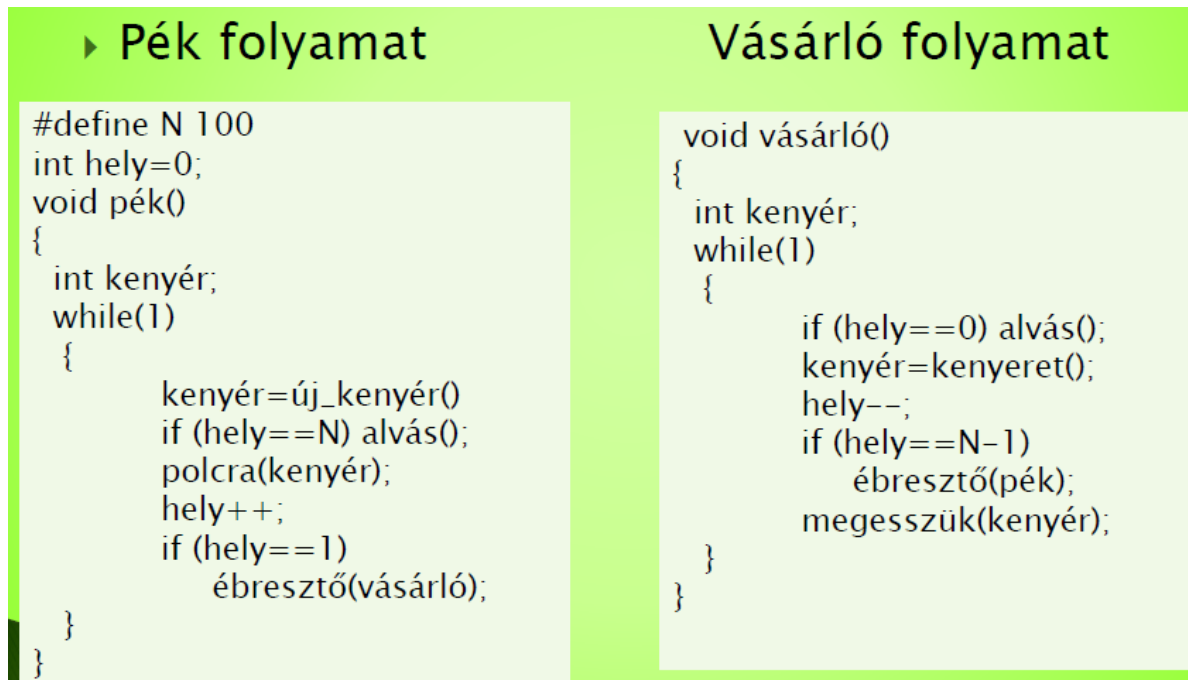


Figure 5: An implementation of the producer-consumer problem

With this implementation a problem can be that access to the “space” variable is not restricted, so this can cause a race condition.

- The customer sees that the space is 0 and then the scheduler hands over the control to the baker, who bakes a bread. Then he sees that the space is 1, sends a wake-up to the customer. This is lost, because the customer is not yet sleeping.
- The customer gets the scheduling back, read the space earlier, it is 0, goes to sleep.
- The baker, after the first one, bakes the remaining N-1 breads and he too goes to sleep

It can be improved with a wake-up bit, but for several processes the problem does not change.

4 Semaphores, shared memory, message passing

4.1 Semaphores

E.W. Dijkstra (1965) proposed the introduction of this new variable type.

- This is in fact an integer variable.
- The semaphore shows a stop signal if its value is 0. Then the process falls asleep, stops before the stop signal.
- If the semaphore is >0 , the way is clear, we can enter the critical section. Two operations belong to it: if we entered, we decrease the value of the semaphore (down); if we exit, we increase the value of the semaphore (up). These Dijkstra called the P and V operations.

Elementary operation: the checking, modification of the semaphore variable, possible falling asleep is an indivisible operation, it cannot be interrupted. This guarantees that no race condition arises.

If the semaphore denotes a typical railway situation, that is, only 1 train can pass through the signal, the value of the semaphore can then be 0 or 1. This is the *binary semaphore*, or otherwise MUTEX (Mutual Exclusion), and we use it for mutual exclusion.

With a system call, at user level, the atomicity of the operations cannot be ensured. At the beginning of the operation, for example, we disable all interrupts. If there are several CPUs, then such a semaphore can be

protected with the TSL instruction. However, these semaphore operations are kernel-level, system-call operations. The development environments provide them.

▶ Gyártó (pék) függvénye	▶ Fogyasztó (Vásárló) függvénye.
<pre>typedef int szemafor; szemafor szabad=1; /*Bináris szemafor, 1 mehet tovább, szabad a jelzés*/ szemafor üres=N, tele=0; /* üres a polc, ez szabad jelzést mutat */ void pék() /* N értéke a „kenyerespolc” mérete */ { int kenyér; while (1) { kenyér=pék_sütő; down(&üres); /* üres csökken, ha előtte>0, mehet tovább*/ down(&szabad); /* Piszkálhatjuk-e a pékség polcát? */ kenyér_polcra(kenyér); /* lgen, betesszük a kenyeret. */ up(&szabad); /* Elengedjük a pékség polcát. */ up(&tele); /* Jelezzük vásárlónak, van kenyér. */ } }</pre>	<pre>void vásárló() /* vásárló szemaforja a tele */ { int kenyér; while (1) { down(&tele); /*tele csökken, ha előtte>0, mehet tovább*/ down(&szabad); /*Piszkálhatjuk-e a pékség polcát? */ kenyér=kenyér_polcra(); /* lgen, levesszük a kenyeret. */ up(&szabad); /* Elengedjük a pékség polcát. */ up(&üres); /* Jelezzük péknek, van hely, lehet sütni. */ kenyér_elfogyasztása(kenyér); } }</pre>

Figure 6: Solution of the producer-consumer problem with semaphores

Free: protects the bread shelf (store), so that only one process can use it at a time (either the baker or the customer): Mutual exclusion, Elementary operations (up, down).

Full, empty semaphore: synchronization semaphores, the producer should stop if the buffer is full, and the consumer should also wait if the buffer is empty.

For a semaphore, swapping two instructions can also cause a problem.

Monitor: a higher-level, language construct. It can contain procedures, data structures. At a time only one process can be active within the monitor.

4.2 Shared memory

Distributed common memory: Memory sharing between processes running in a network.

We have the possibility to make common the memory areas used by different processes, threads within a program, that is, we can use the same memory part, in “time-sharing” mode. To different program parts, processes, threads “connected” in some way to each other, we connect the same memory area.

4.3 Message passing

The processes typically use two primitives: *Send(target process, message)*, *Receive(source, message)*. These are system calls, not language constructs.

If the sender-receiver are not on the same machine, a so-called acknowledgment message is needed. Thus if the sender does not get the acknowledgment, it sends the message again. If the acknowledgment is lost, the sender resends. The distinguishing of repeated messages happens with the help of a sequence number.

Summary: Creation of temporary storage places (mailbox) at both places. It can be omitted, in which case if there is a receive before a send, the sender blocks, and vice versa. This is called the rendezvous strategy. For example Minix 3 also uses rendezvous, with fixed-size messages.

Pipe communication is similar, only in the pipe there are no message boundaries, there is only a byte sequence.

Message passing is a general technique of parallel systems. E.g. MPI

Classic IPC (inter-process communication) problems:

- The dining philosophers case: in a circle, alternately, 5 forks, plates. 2 forks are needed for eating the spaghetti. They compete for the forks next to the plate. They eat and think. The simplest solution (in an infinite loop thinks, picks up its two forks one after another, eats, puts down the forks) hides a few errors, that e.g. there can be a deadlock if they simultaneously grab the left fork and all wait for the right one. And if it puts down the left fork and tries again, that too is not ideal, since they continuously pick up the left fork, then put it down. (Starvation)

```

Int N=5;
Szemafor villa[]={1,1,1,1,1}; //mind
szabad
Szemafor max=4; //max 4 villa használt
//egyszerre
Void filozófus(int i)
{
    while(1)
    {
        gondolkodom();
        down(max);
        down(villa[i]); // bal villa
        down(villa[(i+1)%N]); //jobb
        eszem();
        up(villa[i]);
        up(villa[(i+1)%N]);
        up(max);
    }
}

```

Figure 7: Dining philosophers, improved solution: there are 5 fork semaphores, and a maximum. An example of acquiring a limited resource

- Reader-writer problem: A database can be read by several at once, but only 1 process can write it.

```

// író folyamat
Szemafor database=1;
Szemafor mutex=1;
int rc=0;
Void író()
{
    while(1)
    {
        csinál_valamit();
        down(database); // kritikus
        írunk_adatbázisba();
        up(database);
    }
}

Void olvasó()
{
    while(1)
    {
        down(mutex);
        rc++;
        if (rc==1) down(database);
        up(mutex);
        olvas_adatbázisból();
        down(mutex); // kritikus
        rc--;
        if (rc==0) up(database);
        up(mutex);
        adatot_feldolgozunk();
    }
}

```

Figure 8: Implementation of the reader-writer problem

5 Scheduling options of input and output devices, deadlocks

5.1 Scheduling options of input and output devices

Input-Output devices:

- **Block devices.** We store the information in blocks of a given size. Block size between 512 byte - 32768 byte. They can be written or read independently of each other. They are addressable block by block. Such a device: HDD, tape unit
- **Character devices.** Not addressable, the “characters” (bytes) just come and go in sequence
- **Timer:** an exception, neither block nor character

Interrupts: Generally the devices have a status bit, indicating that the data is ready.

Interrupt use (IRQ):

1. Interrupting the CPU activity
2. Execution of the server with the requested sequence number
3. Reading in the wanted data, performing the closely associated activity

4. Return to the state before the interrupt.

Direct memory access (DMA): Direct Memory Access; it contains a memory address register, a register for the transfer direction indication, for the amount. These can be accessed through regular in, out ports.

The steps of operation:

1. The CPU sets up the DMA controller. (The registers.)
2. The DMA requests the disk controller for the specified operation.
3. After the disk controller has read into its buffer, through the system bus it writes/reads the data to/from the memory.
4. The disk controller acknowledges that the fulfillment of the request is ready.
5. The DMA signals with an interrupt that it finished the operation.

5.2 Deadlocks

Two or more processes, during the acquisition of a resource, get into a situation where they block each other in the further execution. Precise definition: A set of processes is in deadlock if every process waits for an event that can be caused only by another process of the set. It is not tied only to I/O devices, e.g. parallel systems, databases, etc.

According to Coffman E.G., 4 conditions are necessary for a deadlock to arise:

1. *Mutual exclusion condition*. Every resource is assigned to 1 process or is free.
2. *Hold and wait condition*. A process holding a previously obtained resource can request another.
3. *No-preemption condition*. A resource cannot be taken from a process; only the process can release it.
4. *Circular wait condition*. The formation of two or more process chains in which every process waits for a resource that is held by another.

The processes and resources can be modeled with a directed graph. If there is a cycle, then it means a deadlock.

Strategies in case of deadlock:

1. Ignoring the problem.
 - We do not care about it; with high probability it does not find us either.
 - This method is often also known as the ostrich algorithm.
 - According to studies, the ratio of the deadlock problem and other (compiler, OS, hw, sw error) crashes is 1:250.
 - The Unix, Windows world also uses this “method”. But the price is too high for the expected benefit.
2. Detection and recovery.
 - We let the deadlock appear (cycle), we notice this and act.
 - We continuously monitor the resource requests, releases.
 - We continuously manage the resource graph. If a cycle is created, then we terminate a process of the cycle.
 - Another method: we do not deal with the resource graph; if a process has been blocked for x (half an hour?) time, we simply terminate it. This is a known method in mainframe systems.
3. Prevention.

- Thwarting one of the 4 necessary conditions.
- There is always a restriction on one of Coffman's 4 conditions.
 - Mutual exclusion. If a single resource is never assigned exclusively to 1 process, then there is no deadlock either. But this is cumbersome, while e.g. for printer use the printer daemon solves the problem, but here too the printer buffer is a disk area, and here a deadlock can already arise.
 - If there can be no situation where a process holding resources waits for a further resource, then there is likewise no deadlock. This can be achieved in two ways: All resource requirements of a process have to be known in advance, or if a process wants a resource, it should first release all that it holds.
 - Coffman's third condition is no-preemption. Avoiding this is quite hard. E.g. during printing it is not fortunate to give the printer to someone else.
 - The fourth condition, the circular wait, can be eliminated more easily. A simple way: Every process can hold only 1 resource at a time. Another method: We number the resources, and the processes can request the resources only in this order. This is a good avoidance method, only there is no appropriate order.

4. Dynamic avoidance.

- Allocation of resources only "cautiously".
- Banker's algorithm (Dijkstra, 1965). Like the lending practice of a small-town banker. The banker's algorithm, when every request appears, looks at whether fulfilling the request leads to a safe state. If yes, it approves it; if not, it postpones the request. Originally designed for 1 resource. The earlier prevention, and this avoidance too, ask for information (the exact resource requirements, the number of processes in advance) that is hard to give. (Processes are created dynamically, resources change dynamically.) Therefore in practice few apply it.
- Safe states are situations from which there exists an initial state sequence whose result is that every process gets the wanted resources and finishes.

6 Memory management, the concept of virtual memory

Monoprogramming: The simplest memory management method; in time we run only a single program.

Multiprogramming: multiprogramming with fixed partitions: Then the available memory is divided into several, generally not equal-length parts. For its operation every partition needs a so-called waiting queue. If a request arrives, the operating system puts it into the waiting queue of the smallest-size part, called a partition, into which it still fits. In this case that part of the partition is lost – cannot be used – which the currently running process does not use. If the work in the given slice finishes, the longest-waiting one gets the area, and starts operating.

6.1 Memory management

The memory manager is part of the operating system, often in the kernel. Its task: keeping track of the memory, which are free, occupied; allocating memory for processes; releasing memory; controlling the swap between the RAM and the (Hard)Disk.

Two kinds of algorithm groups:

- Moving, swapping of the processes between the memory and the disk is necessary. (swap)
- This is not necessary, if there is enough memory.

Multiprogramming with fixed memory slices: Let us divide the memory into n (not equal) slices. (Fixed slices) E.g. at system boot this can be done. There is one common waiting queue, a separate waiting queue for each slice. A typical solution of batch systems.

Relocation: We do not know where a process gets placed, so the memory references cannot be compiled to fixed values.

Protection: It is undesirable for a program to “access” the memory of another.

Another solution: Use of a base+limit register; these the programs cannot modify, but at every address reference there is a check: slow.

Multiprogramming with memory swap usage: A typical solution of the earlier batch systems is the use of fixed memory slices (IBM OS/MFT). For time-sharing, graphical interfaces this is not ideal. Here we move the whole process between the memory and disk. There is no fixed memory partition, each one changes dynamically, as the OS shuffles the processes back and forth. The system becomes dynamic, with better memory utilization, but the many swaps create holes, therefore memory compaction has to be performed. (In many cases this time loss is not permissible.)

Dynamic memory allocation: It is generally not known how much dynamic data, stack area a program needs. The “code” part of the program gets a fixed slice, while its data and stack part a variable one. These can grow (shrink). If the memory runs out, then the process stops, waits for continuation, or gets to the disk so that the other still-running processes can get memory. If there is an already-waiting process in the memory, that too can get swapped.

The tracking of the “dynamic” memory:

- Definition of the allocation unit. Its size is a question. If small, then the memory holes less, but the tracking “resource (memory) requirement” is large. If the unit is large, then there will be too much memory loss arising from the remainders within the unit.
- Realization of the tracking: With bitmap use or with linked list use. If a process finishes, then it can be necessary to merge the adjacent memory holes. Separate list for the list of holes and processes.

Memory allocation strategies: For a process newly brought in from a new or swap partition, several memory placement algorithms are known (similar to disk):

- First Fit (to the first place where it fits, fastest, simplest)
- Next Fit (the search starts not from the beginning, but from the previous finishing point, less efficient than first fit)
- Best Fit (slow, produces many small holes)
- Worst Fit (there will not be many small holes, but it is not efficient)
- Quick Fit (hole list by size, merging the holes is costly)

6.2 The concept of virtual memory

Multiprogramming with virtual memory usage: A program can use more memory than the available physical size. The operating system keeps only the “necessary part” in the physical memory. A program resides in the “virtual memory space”. The principle can also be used in a monoprogramming environment.

Memory Management Unit (MMU): The virtual address space is divided into “pages”. (This we call the page table). There is a presence/absence bit. We map virtual-physical pages. If the MMU sees that a page is not in the memory, it causes a page fault, the OS frees up a page frame, then brings in the necessary page.

7 Paging and segmentation

7.1 Paging

Paging design considerations:

- Working set model

- Loading the necessary pages. (At start – prepaging) Keeping in physical memory those pages of the process that it uses. This changes with time.
- The pages have to be tracked. If a page has not been referenced in the last N time units, in case of a page fault we throw it out.
- Improvement of the clock algorithm: Let us examine whether the page is an element of the working set? (WSClock algorithm)
- Local, global allocation
 - At a page fault, whether we examine all processes (global), or only the pages belonging to the process (local).
 - In the case of a global algorithm we can provide every process with a page corresponding to its size, which we then change dynamically.
 - Page Fault Frequency (PFF) algorithm, page fault/second ratio; if there are many page faults, we increase the number of in-memory pages of the process. If there are many processes, we can even take a whole process to disk. (load balancing)
- Determination of the correct page size.
 - Small page size: The “page loss” is small, but a large page table is needed for the tracking.
 - Large page size: Conversely, “page loss” is large, small page table.
 - Typically: the page size is $n \times 512$ bytes; for XP, Linuxes the page size is 4KB. 8KB is also used (servers).
- Shared memory
 - We can allocate a memory area that several processes can use.
 - Distributed common memory: Memory sharing between processes running in a network.

7.2 Segmentation

Virtual memory: a one-dimensional address space, from 0 to the maximum address (4, 8, 16 GB, ...).

Several programs have a dynamic area, which can grow, with uncertain size. Let us create address spaces independent of each other; these we call segments.

In this world an address consists of 2 parts: segment number, and the address within it. (offset)

Segmentation makes the “simple” realization of shared libraries possible. The program can be logically split apart, data segment, code segment, etc. We can also specify a protection level for a segment. Pages are fixed-size, segments are not. Appearance of segment fragmentation. This can be improved with fragmentation-compaction.

8 Page replacement algorithms

If a virtual-address page is not in the memory, then a page has to be thrown out, and this new page brought in. The situation is similar with processor cache memory usage, or with the browser’s local cache. Optimal page replacement: Let us label every page with the number indicating how many CPU instructions are executed before we reference it. Let us throw out that page in which this number is smallest. There is one problem, it cannot be realized, but in a double run it can serve testing purposes.

- NRU (Not Recently Used) algorithm:
 - Let us use the modify (Modify) and reference (Reference) bit of the page table entry. Let us set the reference bit to 0 from time to time (at a clock interrupt, about 0.02 sec); with this we indicate whether it was used, referenced “in the recent time”.
 - * Class 0: not referenced, not modified

- * Class 1: not referenced, modified. One can get here if the clock interrupt sets the reference bit to 0.
- * Class 2: referenced, not modified
- * Class 3: referenced, modified
- Let us choose a page randomly from the smallest non-empty class. Simple, not really efficient to implement, gives an appropriate result.
- FIFO page replacement. Simple FIFO, also known from other areas; if a new page is needed, then we throw out the oldest page. In a list the pages in the order of arrival, a page arrives at the beginning of the list, and departs from the end. Its improvement is the Second Chance page replacement algorithm.
- Second Chance page replacement algorithm. Like FIFO, only if the reference bit of the page at the end of the list is 1, then it gets a second chance, gets to the beginning of the list, and we set the reference bit to 0.
- Clock algorithm: like the second chance, only let us not move the pages around in a list, but place them in a circle and go around with a pointer. The pointer points to the oldest page. At a page fault, if the reference bit of the pointed page is 1, we zero it, and examine the next page. If the reference bit of the examined page is 0, then we throw it out.
- LRU (Least Recently Used) algorithm: Throwing out the least (longest ago) used page. HW or SW realization.
 - HW1: Take a counter that increases by 1 at every memory reference. We can store this counter in every page table. At every memory reference we write this counter into the page. At a page fault we find the page with the smallest counter value.
 - HW2: With LRU bit matrix usage, n pages, $n \times n$ bit matrix. At a reference of the k -th page frame let us set the k -th row of the matrix to 1, and its k -th column to 0. At a page fault the smallest-value row is the oldest.
- NFU (Not Frequently Used) algorithm:
 - Let us put a counter to every page. At every clock interrupt let us add to this the reference (R) bit of the page.
 - At a page fault let us throw out the page with the smallest counter value. (The most not-used page)
 - An error is that NFU does not forget; pages frequently used at the beginning of a program preserve their high value.
 - Let us modify it: At every clock interrupt let us do a bit shift to the right on the counter, and from the left let us put in the reference bit (shr). (Aging algorithm)
 - This approximates the LRU algorithm well.
 - This page counter has a finite number of bits (n), so events before n time units it certainly cannot distinguish.

9 Disk space organization, redundant arrays, services and typical implementations of file systems

9.1 Disk space organization

9.2 Redundant arrays

RAID – Redundant Array of Inexpensive Disks. If the operating system provides it, it is often called Soft Raid. If an intelligent (external) controller unit provides it, it is often called Hardware Raid, or just a Raid disk system.

Although “Inexpensive” is in the name, in reality it is rather not. It joins several disks together, and the operating system sees it as a single logical unit. Several kinds of “joining” principle exist: RAID 0-6.

- RAID 0 (striping)
 - This is the Raid that is not even redundant
 - By the logical concatenation of several disks we get one drive.
 - The sum of the disk capacities gives the capacity of the new drive.
 - It distributes the blocks of the logical drive onto the disks (striping), thereby the writing of one file gets onto several disks.
 - Faster I/O operations.
 - No protection against failure.
- RAID 1 (mirroring)
 - It makes one logical unit from two independent disks.
 - It writes out every piece of data in parallel to both disks. (Mirroring, mirror)
 - Storage capacity decreases to half.
 - An expensive solution.
 - Significant fault-tolerance capability. The simultaneous failure of both disks causes data loss.
- RAID 1+0, RAID 0+1
 - RAID 1+0: Let us join together several of mirrored disks.
 - RAID 0+1: Let us take two of a RAID 0 joined disk group.
 - The controllers often provide one or the other, since either way there is mirroring, that is, it is expensive, so it is rarely used.
- RAID 2: Besides the data bits it also contains error-correcting bits. (ECC: Error Correction Code) E.g. 3 correcting disks for 4 disks
- RAID 3: A single extra “parity disk” is enough, $n+1$ disks, $\sum n$ is the capacity
- RAID 4: Supplementing RAID 0 with a parity disk.
- Today these solutions (RAID 2, 3, 4) are not frequently used.
- RAID 5
 - There is no parity disk, this is distributed over all elements of the array. (stripe set)
 - The data is also stored distributed.
 - Intensive CPU requirement (controller CPU!!!)
 - Redundant storage, the failure of 1 disk does not cause data loss. The lost one can be computed from the parity bit and the others. The simultaneous failure of 2 disks already causes data loss.
 - N disks in a RAID 5 array ($N \geq 3$) give a logical drive of size $n-1$ disks.
- RAID 6
 - To the RAID 5 parity block, an error-correcting code is stored. (+1 disk)
 - Even more intensive CPU requirement.
 - The simultaneous failure of two disks also does not cause data loss.
 - Relatively expensive

- The capacity of a RAID 6 array of N disks is identical to the capacity of N-2 disks.
- In principle the method can be generalized (failure of 3 disks)

Today most often the RAID 1, 5 versions are used. The RAID 6 controllers appeared in the last 1-2 years. Although RAID is about inexpensive disks, in reality these are not always inexpensive. With RAID 6, 2 disks already drop out, so this is even more expensive.

Hot-Swap RAID controller: during operation we simply replace the failed disk.

9.3 Services of file systems

File: a logical group of data, provided with a name and other parameters. The file is the unit of information storage; we refer to it by name. Typically it is located on a disk, but generally the data set, data stream can be tied even to a screen, keyboard.

On the disk there are generally 3 kinds of file:

- Regular user file.
- Temporary file
- Administrative file. This is necessary for operation, generally hidden.

Directory: a logical grouping of files (directories).

File system: a method for forming the system of placement of files and directories on our physical disk, volume.

Placement of files: At the beginning of the partition, the so-called Superblock (e.g. for FAT, block 0) describes the characteristics of the system. Generally the space tracking (FAT, linked-list tracking) follows. After this the directory structure (inode), with the directory entries, file data. (For FAT16 the directory is first, then after it the file data.)

Placement strategies:

- Contiguous space allocation: First Fit, Best Fit, Worst Fit (we put it into a memory section so that the largest possible part stays free). Wasteful disk utilization.
- Linked placement: No loss (only that arising from the block size). File data (broken into blocks) linked-list table. The reading of the n-th block will be slow. Free-occupied sectors: File Allocation Table, FAT. This can be large; the FAT has to be in the memory for a file operation.
- Index-table placement: The catalog contains the address of the small table (inode) belonging to the file. From an inode address the file is reachable.

File system types:

- File system applied on a hard disk: FAT, NTFS, EXT2FS, XFS, etc
- File system applied on tape systems (primarily backup): Table of contents, then the content sequentially
- CD, DVD, Magneto-opto Disc file system: CDFS, UDF (Universal Disc Format), compatibility
- RAM disks (today less used)
- FLASH memory drive (FAT32)
- Network drive: NFS
- Other pseudo file systems: Zip, tar.gz, ISO

Journalized file systems: The file system can get into an inconsistent state in case of damage, power outage, etc. Often called: LFS (Log-structured File System) or JFS (Journalized). On the pattern of database managers: operation + log is journaled. Built on a transaction basis. In case of a shutdown, error, it can be recovered based on the log. Practically the log is another disk (another partition). Higher resource requirement, but greater reliability.

Today's operating systems support "a lot of" types, e.g.: the Linux 2.6 kernel more than 50. The file system can be mounted in several ways:

- Mount, as a result of which the file system files become accessible.
- Automatic mounting (e.g. USB drive)
- Manual mounting (Linux, mount command)

Separate-namespace accessibility for Windows is the A, B, C, etc. disks. A unified namespace is in UNIX.

9.4 Typical implementations of the services of file systems

- FAT
 - File Allocation Table. Perhaps the oldest file system still alive today.
 - The FAT table is the occupancy map of the disk; it has as many elements as there are blocks on the disk. E.g.: Fat12, FDD, Cluster size 12 bits. If its value is 0, free; if not, occupied. For safety there are 2 tables.
 - Linked placement. In the catalog, besides the file data (name, etc.), only the sequence number of the first file block is given. The FAT block identifier shows the address of the next block. If there is no more, the value is FFF.
 - Fixed entry size, 32 bytes (max. 8.3 name)
 - System, Hidden, Archive, Read only, directory attributes
 - The time of the last modification of the file is also stored.
 - *FAT16*, 16-bit cluster descriptor, 4 bytes (2x2) describe the start block of the file. Max. 4 GB partition size (with 64kb block size), typically 2 GB. The maximum file size is also 4 (2) GB. Separate directory area (for FDD, track 0). On FDD 512 directory entries. On HDD 32736 directory entries (16-bit signed)
 - *FAT32* (available from 1996): 28-bit cluster descriptor, 2 TB partition size (with base sector size), up to 32 MB 1 block = 1 sector (512 bytes). 64 MB: 1 block = 1KB (2 sectors), 128MB: 1 block = 2KB. 1 block can be max. 64 KB.
 - It also already supported long file names. With multiple entries reserved for 8.3 parts.
 - Defragmentation is necessary.
- NTFS
 - New Technology File System
 - FAT-NTFS efficiency boundary: about 400 MB.
 - 255-character file name, 8+3 secondary name
 - Sophisticated security settings
 - As with FAT, defragmentation is also necessary here.
 - Support of encrypted file system, journaling
 - POSIX support. Hardlink (fsutil command), timestamps, upper/lower case differ
 - Compressed file, folder, user quota management

- NTFS only tracks clusters, not sectors (512 bytes)

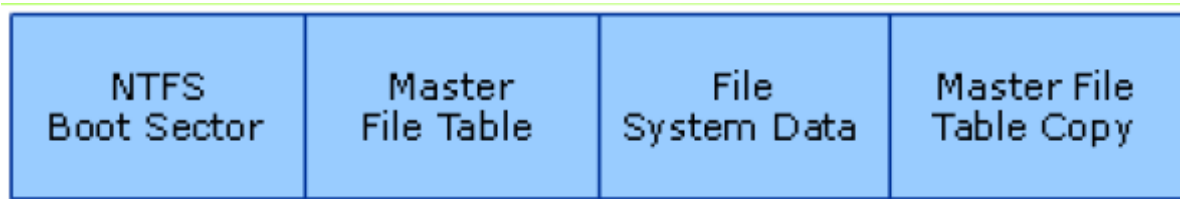


Figure 9: NTFS partition. The Master File Table and the File System Data are each a table

- MFT: An NTFS partition begins with the MFT (Master File Table) table. 16 attributes give one file entry. Every attribute is max. 1kb. If this is not enough, then one attribute points to the continuation. The data is also a kind of attribute, so one entry can contain several data rows. (E.g.: Preview image) The theoretical file size can be 2^{64} bytes. If the file < 1kb, it fits into the attribute, a direct file. There is no maximum file size.

0	\$Mft – Master File Table
1	\$MftMirr – MFT Mirror
2	\$LogFile – Naplófájl
3	\$Volume – Kötetfájl
4	\$AttrDef – Attribútum definíciók
5	\ – Gyökérkönyvtár
6	\$BitMap – Cluster foglaltság
7	\$Boot – Bootszektor
8	\$BadClus – Hibás clusterek
9	\$Secure – Biztonsági leírók
10	\$UpCase – Unicode karaktertábla
11	\$Extend – Egyéb metadata
12	Nem használt
...	...
15	Nem használt
16	Felhasználói fájlok és mappák

Az NTFS metadata számára fenntartva

Figure 10: The structure of the NTFS partition

- ext, ext2 and ext3

- The term “ext” in the names of the file systems covers the term extended. The extended file system was the first file system made specifically for UNIX-like GNU/LINUX operating systems, which inherited the metadata structure of the UFS (UNIX File System) file system, and was made to eliminate the errors of the Minix operating system’s file system. The elimination of the errors is, among other things, the extension of the file system boundaries of the Minix operating system.
- The ext2 file system, which appeared on systems other than GNU/LINUX operating systems too, was the default file system of several Linux distributions, until its successor, the “ext3” file system (third extended filesystem) was made.

- The ext3 file system (third extended filesystem) is the successor of the ext2 file system, which compared to the ext2 file system already also contains journaling. This journaling primarily increases security, and makes it possible that after an irregular shutdown the whole file system does not have to be re-checked.
- ReiserFS: The ReiserFS file system makes it possible to store variable-size files on a block device and organize them into a directory structure. The early UNIX and UNIX-like operating systems (such as e.g. the GNU/LINUX operating system too) supported only one kind of file system, their own format. Modern operating systems, however, support several kinds of file systems, and there are also file systems that several operating systems support. The ReiserFS file system is not at all such. The ReiserFS file system is a file system that currently, without restriction, can only and exclusively be used under the GNU/LINUX operating system.